

1. Problems with containers

Containers revolutionized the way applications are packaged and shipped. They made it easy to run the same application across different environments without configuration issues. But in real industry conditions—large user traffic, multiple teams, frequent updates, and distributed systems—containers alone are not enough. They introduce a new set of challenges that become unmanageable when applications grow.

Problem with Containers and how container orchestration solves them?

1. Containers Multiply Very Quickly:

- > In real applications, traffic is unpredictable. During peak hours, an application that normally uses three containers may suddenly require twenty or fifty.
- > Without orchestration, you must manually start and stop containers and track which container is running on which server. This becomes impossible when the number grows.

Example:

- > An e-commerce website launches a flash sale. The checkout service suddenly needs 40 containers instead of 5. Manually launching and tracking these containers becomes chaotic and error-prone.

2. Manual Deployment Is Slow and Risky:

- > Running a container isn't just "docker run." Real applications require environment variables, ports, volumes, networks, and secrets. Managing these manually for dozens of containers leads to mistakes.

Example:

- > A developer launches a new version of the payment service but forgets to set one environment variable. The entire payment flow breaks in production.

3. Containers Crash and Disappear:

- > Containers are lightweight; if something goes wrong, they simply exit. Without automation, this causes outages.

Example:

- > A Flask backend crashes at midnight due to memory leak. Nobody notices until morning. Users see errors for hours.

4. No Built-In Load Balancing:

- > Containers cannot share incoming traffic by themselves. If one container gets too much load, it becomes slow or crashes.

Example:

- > Your website runs five NGINX containers. All users accidentally hit only the first container because there is no load balancer. That container gets overloaded and fails.

5. Updates and Rollbacks Are Hard:

- > Deploying a new version often means stopping old containers and manually starting new ones. If something goes wrong, rolling back is slow.

Example:

- > You stop v1 containers and deploy v2. It has a bug. Rolling back takes 10–15 minutes. During this time, users experience downtime.

6. Networking Between Containers Is Complicated:

- > Containers get new IP addresses every time they restart. Without proper networking management, services cannot find each other.

Example:

- > Order Service calls Payment Service using its IP. Payment container restarts and gets a new IP. As a result, all payments fail.

This is where container orchestration platforms like Kubernetes, Docker Swarm, and Amazon ECS step in. They operate like an automated "control system" that manages containers at scale.

What is Container Orchestration?

- > Container Orchestration is the automated process of deploying, managing, scaling, networking, and monitoring containers so that applications run reliably and efficiently in production.
- > In simple words, it is a system that automatically handles everything needed to keep containerized applications running smoothly — including starting containers, restarting them if they fail, distributing traffic, adding more containers during high load, updating them safely, and connecting them with each other.

How Orchestration Solves It:

- > Orchestration decides where and how many containers should run. You simply say "run 40 containers," and the system automatically distributes them across available machines.
- > You define everything (images, ports, variables, storage) in a Deployment/Task definition. The orchestrator automatically pulls the image, applies configs, and starts containers the same way every time.

- > Orchestration continuously monitors containers. If one fails, it restarts or replaces it immediately. This is known as self-healing.
- > Orchestrators create a Service or Load Balancer that automatically routes requests to all active containers evenly. If one container dies, traffic is redirected instantly.
- > Orchestration supports rolling updates and instant rollbacks. It replaces containers gradually and if errors increase, it automatically switches back to the previous version.
- > Orchestrators provide service discovery. Instead of calling by IP, Order Service calls "payment-service," which always points to the correct container no matter how many times it restarts.

Container orchestration platforms act as the automated brain responsible for managing everything containers cannot do by themselves. They automatically deploy, monitor, scale, network, load-balance, and heal containers. They support safe updates, secure secrets, persistent storage, and rollbacks. In short, orchestration converts a fragile containerized system into a highly reliable, self-managing, production-grade environment.

Container Orchestration tools:

Container orchestration tools automate the entire lifecycle of containers. They handle tasks such as:

- > Deploying containers
- > Distributing them across machines (nodes)
- > Monitoring their health
- > Restarting them when they fail
- > Scaling up or down based on load
- > Performing rolling updates without downtime
- > Managing networking between services
- > Managing storage for containers

Container Orchestration tools:

1. Kubernetes (K8s)
2. Amazon ECS
3. Amazon EKS
4. Docker Swarm
5. Nomad (HashiCorp)
6. Azure AKS / Google GKE (managed Kubernetes services)

What is Kubernetes?

- > Kubernetes (often called K8s) is an open-source container orchestration platform originally created by Google. Its purpose is to automatically manage containers running across multiple servers.
- > Kubernetes ensures that applications always run in the desired state. For example, if you say you want 5 containers running, Kubernetes keeps 5 running at all times. If one container dies, Kubernetes replaces it automatically.

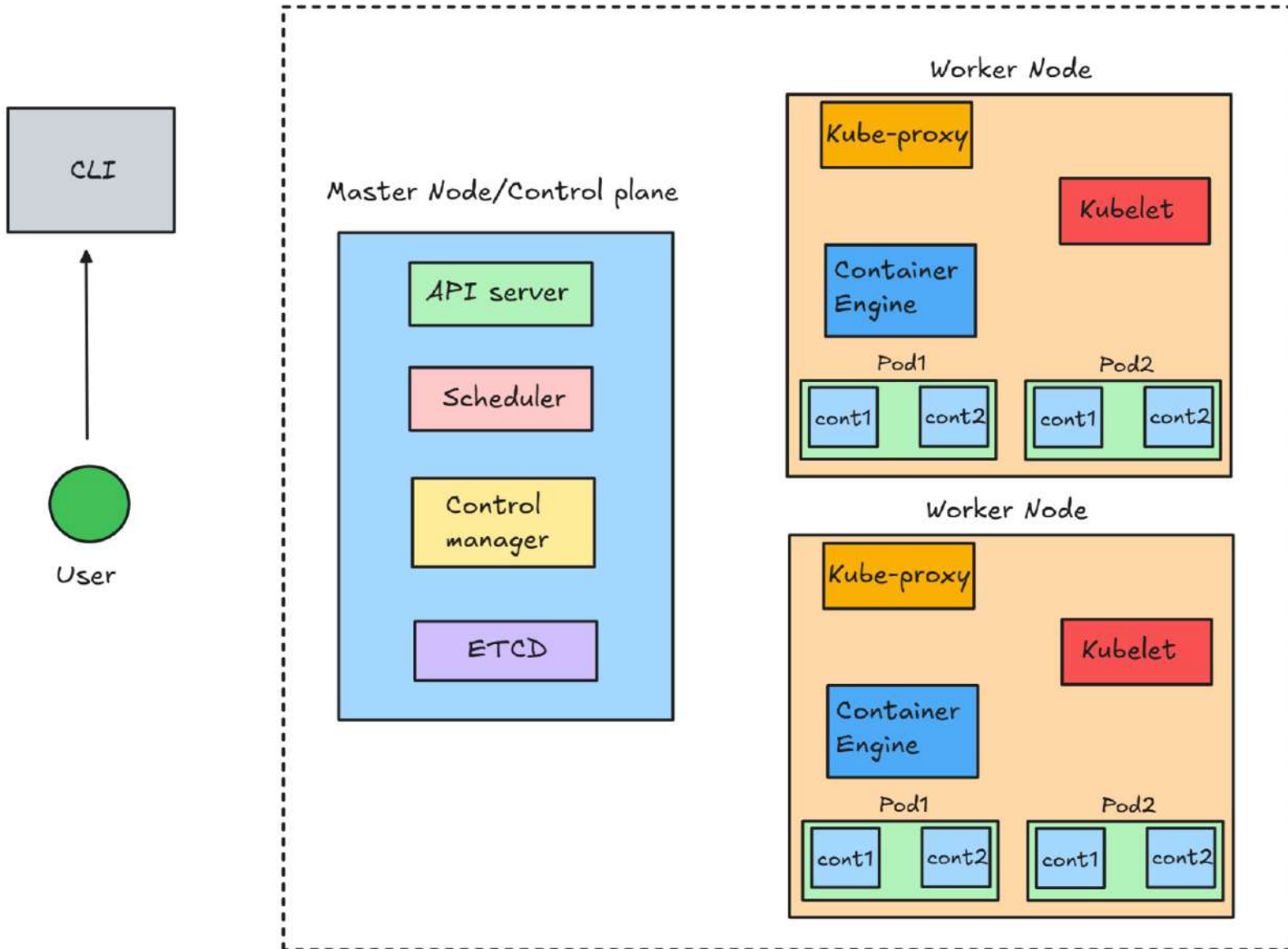
Some major features include:

- > Automatically restarts failed containers.
- > Automatically reschedules containers on healthy machines.
- > Can scale up or down based on CPU, memory, or custom metrics.
- > Provides built-in service discovery and networking.
- > Supports rolling updates and instant rollbacks.
- > Works on any cloud provider or even on-premises.

History:

- > Google was using internal system called as "Borg" which is used for running large scale containerized application.
- > Google decided to create open source container orchestration platform and in the year 2015 they launched Kubernetes as free and open source tool.
- > In the year 2016, CNCF (Cloud native computing foundation) registried kubernetes as free and open source.

CLUSTER



Here is the Kubernetes architecture flow:

1. A user sends a request (like creating a Pod) using `kubectl` or an API call.
2. The API Server receives the request and validates it.
3. The API Server writes the desired state (Pod definition) into `etcd`, the cluster's database.
4. The Controller Manager constantly watches `etcd` to see what the desired state is.
5. When the Controller Manager notices a new desired object (e.g., a Pod), it decides that action is needed.
6. The Controller Manager informs the Scheduler that a Pod needs to be placed on a node.
7. The Scheduler evaluates all nodes and chooses the best one based on CPU, RAM, taints, affinity, etc.
8. The Scheduler sends the scheduling decision back to the API Server.
9. The API Server notifies the Kubelet on the chosen node.
10. The Kubelet pulls the required container images and creates the Pod using the Container Runtime (containerd, Docker, CRI-O).
11. The Kubelet continuously monitors the Pod and sends status updates back to the API Server.
12. The Kube-Proxy on the node sets up networking, gives the Pod an IP address, and configures routing rules.
13. If the Pod crashes, the Controller Manager detects the mismatch between desired and actual state.
14. The Controller Manager instructs Kubernetes to recreate the Pod on the same or a different node.
15. The API Server updates `etcd` with the new actual state of the cluster.
16. This cycle continues constantly, ensuring Kubernetes maintains the desired state across the cluster.

When a Pod dies, here is exactly what happens:

1. Kubelet reports to the API Server that the Pod is no longer running.
2. The API Server updates etcd — actual state is now "Pod is dead."
3. The Controller Manager notices the mismatch between:
 - > Desired state (Pod should exist)
 - > Actual state (Pod is dead)
4. The Controller Manager does NOT directly talk to the Scheduler. Instead, it updates the API Server by creating a new unscheduled Pod object that needs placement.
5. The Scheduler watches the API Server and sees the new Pod that has no node assigned.
6. The Scheduler selects a node and sends that decision to the API Server.
7. The API Server informs the Kubelet on the chosen node.
8. The Kubelet recreates the Pod using the container runtime.

The Controller Manager communicates only with the API Server.

The Scheduler also communicates only with the API Server.

They never talk to each other directly.